



後端工程師前置作業 Backend Engineer Assessment

嗨 🙌 謝謝你投遞了 AmazingTalker Backend Engineer 系列的職缺
完成後請將程式碼打包並透過[這份表單](#)上傳給我們

Requirements & Recommendations

- 共有 4 題題目，請依照順序使用 90 分鐘完成
- 每個題目的要求不同，請依照題目本身的敘述來完成
- 90 分鐘很有可能沒辦法完美完成每個細節，請優先思考怎麼表達自己的想法
- 我們會專注在作業中表達出來的思考脈絡，正確性、完整度並不會是這些題目的首要考量

⚠️ 請注意資料夾案檔案命名，避免自動化流程的 bot 轉發作業時遺失。

上傳的資料夾結構請依照下方規則：

```
{date}_{your_name}/ <-- 20210101_AndrewChen_陳小安 |— 001.  
{your_ext} <-- your_ext 請更換成與內容一致的副檔名 |— 002.{your_ext}  
|— 003.txt |— 004.go
```

Base 7 converter

Given an integer, return its base 7 string representation.

Free to use any programming language. **Please write the function yourself instead of using the built-in one.**

Example 1:

```
Input: 20 Output: "26"
```

Example 2:

```
Input: -7 Output: "-10"
```

002

Two Sum Less Than K

Given an array A of integers and integer K. Any integer pair in A could be combined as a set. Find and return all combinations of sets where the sum S of elements is the maximum but less than K (a, b are in A and $a + b = S < K$ and S is maximum, return [a, b]). The answer must exclude duplicates, but it's fine in any order. e.g. [1, 2] or [2, 1] may be treated as the same, no need to return twice in one output. Please answer the time complexity of your program, too.

Free to use any programming language

Example 1:

```
Input: A = [1, 2, 3, 4]. K = 4 Output: [1, 2], because 1+2 = 3 < 4
```

Example 2:

```
Input: A = [1, 2, 3]. K = 3 Output: []
```

Example 3:

```
Input: A = [1, 2, 2, 3, 4]. K = 5 Output: [[1, 3], [2, 2]], because both `1+3` and `2+2` are 4 < 5
```

003

Unite test

Whenever a new commit submits in GitHub. We always run the CI process to build up the docker image for the server. At the same time. We run test cases to make sure all functionality fine. Please design unit test cases for the following "TeacherDataAccessObject". It's OK to write down the cases what you are going to verify without implementation. There is no any constraint on the format. We will follow up the detail in the interview soon.

► Answer Demo

```
package main import ( "time" "github.com/jinzhu/gorm" ) //
===== DO NOT EDIT ===== //var db gorm.DB //
connected db client //var cache redis.Client // connected redis
client //var logger zap.Logger // some useful logger for all
developers in company type MessageName string const (
TeacherCreated MessageName = "teacher_created" ) type Teacher
struct { ID int64 `json:"id"` Name string `json:"name"` DeletedAt
*time.Time `json:"deleted_at"` } type MessageQueue interface {
Publish(MessageName, map[string]interface{}) error } type
TeacherDataAccessObject interface { CreateTeacher(string)
(*Teacher, error) } type TeacherDAO struct { mq MessageQueue db
*gorm.DB } func NewTeacherDAO(mqClient MessageQueue, dbClient
*gorm.DB) TeacherDataAccessObject { return TeacherDAO{ mq:
mqClient, db: dbClient, } } func (dao TeacherDAO)
CreateTeacher(name string) (*Teacher, error) { if name == "" {
return nil, errors.New("name should not be empty") } teacher :=
Teacher{Name: name} r := dao.db.Create(teacher) if r.Error != nil {
return nil, r.Error } payload := map[string]interface{}{
"teacherId": teacher.ID, } err := dao.mq.Publish(TeacherCreated,
payload) if err != nil { // use logger here } return &teacher, nil
}
```

Try to refactor the following code

1. Make it readable and testable
2. Please check error handling issues, maybe some error hidden or no-need to emit
3. We will focus on how you design new functions. It should be scalable and high reusable.

others

1. Assume database client and Redis client already connected, don't worry about it
2. There is no 100% correct answer for this question. Just do your best, we will ask you more during the interview
3. Do not use any new third party go modules

Demo of API usage

```
GET /admin/teachers?page=1&size=30 GET /admin/teachers?
page=1&size=30&deleted=true
```

Code

- Ruby (recommended)

▼ Golang

Go ▾

```
// Demo of API usage: // GET /admin/teachers?page=1&size=30 //
GET /admin/teachers?page=1&size=30&deleted=true package main
import ( "encoding/json" "fmt" "strconv" "time" "github.com/gin-
gonic/gin" redis "github.com/go-redis/redis/v8"
"github.com/jinzhu/gorm" "go.uber.org/zap" ) // ===== DO
NOT EDIT ===== var db gorm.DB // connected db client var
cache redis.Client // connected redis client var logger
zap.Logger // some useful logger for all developers in company
type Teacher struct { ID int64 `json:"id"` Name string
`json:"name"` DeletedAt *time.Time `json:"deleted_at"` } func
cacheMarshaller(value interface{}) ([]byte, error) { return
json.Marshal(value) } func cacheUnMarshaller(val string,
container interface{}) error { return
json.Unmarshal([]byte(val), container) } // ===== DO NOT
EDIT END ===== // =====
func main() { r := gin.Default() r.GET("/admin/teachers",
GetTeachers) r.Run() } func GetTeachers(ctx *gin.Context) {
withDeleted := false page, _ :=
strconv.ParseInt(ctx.Query("page"), 10, 64) size, _ :=
strconv.ParseInt(ctx.Query("size"), 10, 64) if
ctx.Query("deleted") == "true" { withDeleted = true } cacheKey
:= fmt.Sprintf("%d-%d-%s", page, size, withDeleted) cacheResult,
err := cache.Get(ctx, cacheKey).Result() if err != nil && err !=
redis.Nil { ctx.JSON(500, gin.H{"message": err.Error()}) return
} if err == redis.Nil { offset := size * (page - 1) query :=
db.Where("").Limit(size).Offset(offset) if !withDeleted {
query.Where("deleted_at IN NOT NULL") } var teachers []Teacher
err := query.Find(&teachers).Error if err != nil { ctx.JSON(500,
gin.H{"message": err.Error()}) return } if bs, err :=
cacheMarshaller(teachers); err != nil { ctx.JSON(500,
gin.H{"message": err.Error()}) return } else { if err :=
cache.Set(ctx, fmt.Sprintf("%d-%d-%s", page, size, withDeleted),
bs, time.Minute*3).Err(); err != nil { ctx.JSON(500,
gin.H{"message": err.Error()}) return } } ctx.JSON(200,
teachers) } else { var teachers []Teacher if err :=
cacheUnMarshaller(cacheResult, &teachers); err != nil {
ctx.JSON(500, gin.H{"message": err.Error()}) return }
ctx.JSON(200, teachers) } }
```

- Node.js
- Python

